OPERATING SYSTEMS (CSA0403)
Assignment 2 – Task 1 (Q1): Processes and Process States
Problem Context – Online Examination Server

Field	Details
Name	Juhi Surin
Register Number	192525148
Course	CSA0403 – Operating Systems
Assignment	Assignment 2 – Task 1
Date	03-11-2026
Under the Supervision of	Dr. Velmurugan J

1. Problem Context and Scope

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

This report answers every part of the question: the process-state diagram and the process control block (Section 2 and Section 3), a program that creates child processes with fork() observed with ps and /proc (Section 4), the evaluation of where the time goes (Section 5), the design decision (Section 6), and the alternative concept and complexity analysis for threads (Section 7).

Evidence note. The outputs in Section 4 and Section 7 were produced by actually compiling and running the programs listed in this report, in the environment given in Table 1. No test with 2,000 simultaneously active sessions was run; the arguments in Section 5 about 2,000 active processes are reasoned from how the operating system works, and any numbers used there are labelled as assumptions.

Table 1: Environment used for the executed programs

Item	Value
Compiler	gcc 13.3.0
Operating system	Ubuntu 24.04.4 LTS (Linux kernel 6.18.44)
Processors available	1
Compile command	gcc -std=c11 -Wall -Wextra -pthread

2. Process-State Diagram

A process moves between five states during its life. Figure 1 shows the states and the events that cause each transition. Table 2 explains each transition for the examination server.

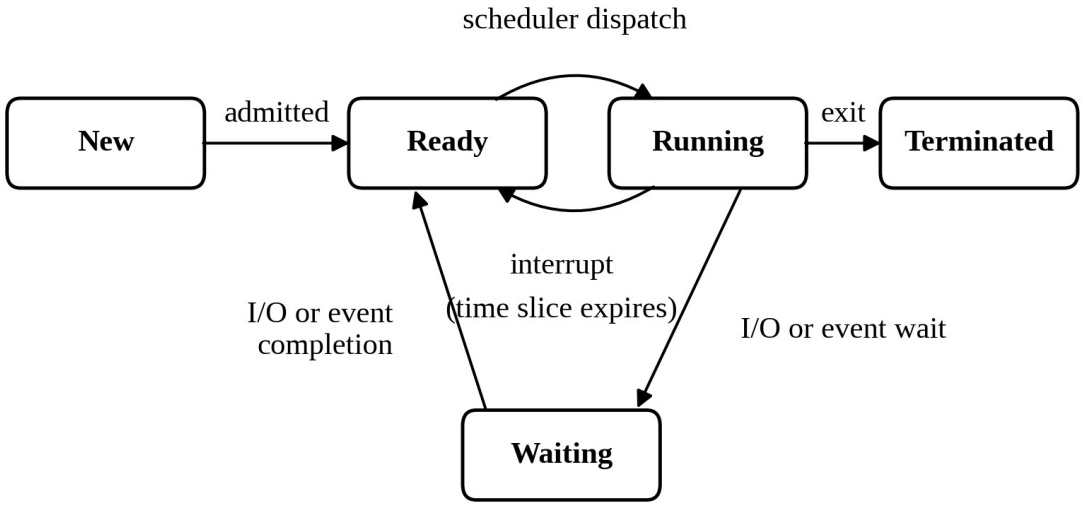


Figure 1: Process-state diagram

Table 2: State transitions in the examination server

Transition	Cause	Example in the examination server
New → Ready	Admitted: the OS has created the PCB and allocated memory.	A student logs in and the server creates the session process.
Ready → Running	Scheduler dispatch: the CPU scheduler selects the process.	The session process is given a CPU core.
Running → Ready	Interrupt: the time slice expires or a higher-priority process arrives.	Another of the 2,000 sessions is given the CPU.
Running → Waiting	The process requests I/O or waits for an event.	The session waits for a question to be read from disk, or for the student's next input.
Waiting → Ready	I/O or the awaited event completes.	The disk read finishes and the session can run again.
Running → Terminated	The process calls exit() or is killed.	The student submits the exam and the session ends.

3. Process Control Block (PCB)

The operating system keeps one process control block for every process. The PCB holds everything needed to stop the process and later resume it exactly where it stopped. During a context switch the kernel saves the running process's registers into its PCB and loads the registers of the next process from that process's PCB. In Linux the PCB is the structure task_struct.

Table 3: Contents of the process control block

PCB field	Contents	Example for one student session
Process identifier	Unique PID, and the PID of the parent process (PPID).	PID of the session process; PPID of the server that created it.
Process state	New, ready, running, waiting or terminated.	Waiting while the session reads an exam question.
Program counter	Address of the next instruction to execute.	Where the session resumes after a context switch.
CPU registers	Accumulators, index registers, stack pointer and flags, saved on every switch.	Register values of the session when it is interrupted.
CPU-scheduling information	Priority, scheduling queue pointers and other scheduling parameters.	Priority of the session relative to the auto-save process.
Memory-management information	Page-table pointer, memory limits and segment information.	Page table of the session's private address space.
Accounting information	CPU time used, time limits and process number.	CPU time consumed by the session.
I/O status information	Open files, allocated devices and pending I/O requests.	The answer file and the network socket of the student.

4. Creating Child Processes with fork(), Observed with ps and /proc

4.1 Program

The parent plays the role of the exam server and creates five children, each standing for one student session. Each child sleeps for three seconds so that it stays alive long enough to be observed, then exits with its child number as the exit status. The parent waits one second, runs ps to list its children, reads /proc/<pid>/status for the first child, and finally collects every child with waitpid(). Five children are used so that the output is easy to read.

Listing 1: fork_demo.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NCHILD 5

int main(void)
{
    pid_t pids[NCHILD];
    char cmd[96];

    printf("parent PID %d creating %d children\n", (int)getpid(), NCHILD);
    fflush(stdout);

    for (int i = 0; i < NCHILD; i++) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); exit(1); }
        if (pid == 0) {          /* child: simulates a student session */
            sleep(3);            /* stays alive for ps and /proc */
            _exit(i);            /* exit status = child number */
        }
        pids[i] = pid;
    }

    sleep(1);                    /* let all children start */

    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,comm --ppid %d",
             (int)getpid());
    printf("--- ps output ---\n");
    fflush(stdout);
    system(cmd);

    snprintf(cmd, sizeof cmd,
             "grep -E 'Name|State|Pid|PPid|VmRSS' /proc/%d/status",
             (int)pids[0]);
    printf("--- /proc/%d/status ---\n", (int)pids[0]);
    fflush(stdout);
    system(cmd);

    for (int i = 0; i < NCHILD; i++) {
        int status;
        waitpid(pids[i], &status, 0);
        printf("child %d (PID %d) exited with status %d\n",
               i, (int)pids[i], WEXITSTATUS(status));
    }
    return 0;
}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra -o fork_demo fork_demo.c
./fork_demo
```

4.2 Output

The output below was produced by running Listing 1 in the environment of Table 1.

Output 1: Running fork_demo

```
parent PID 24246 creating 5 children
--- ps output ---
  PID  PPID STAT COMMAND
24247 24246 S    fork_demo
24248 24246 S    fork_demo
24249 24246 S    fork_demo
24250 24246 S    fork_demo
24251 24246 S    fork_demo
24252 24246 S    sh
--- /proc/24247/status ---
Name: fork_demo
State: S (sleeping)
Pid: 24247
PPid: 24246
TracerPid: 0
VmRSS:      276 kB
child 0 (PID 24247) exited with status 0
child 1 (PID 24248) exited with status 1
child 2 (PID 24249) exited with status 2
child 3 (PID 24250) exited with status 3
child 4 (PID 24251) exited with status 4
```

4.3 Interpretation of the evidence

- **ps:** the five children all show the same PPID, the PID of the parent, which confirms that fork() created them as the parent's children. The last line, sh, is the shell that system() starts in order to run ps itself.
- **STAT column:** the value S means the process is sleeping, that is, in the Waiting state of Figure 1, because each child is inside sleep(). Linux shows R for both Ready and Running, T for stopped and Z for a terminated process that has not yet been collected by its parent (a zombie).
- **/proc/<pid>/status:** the kernel's view of the PCB. It reports Name, State, Pid and PPid (the grep pattern also matches TracerPid), and VmRSS, the physical memory currently used by the process.

- **waitpid():** the parent collects every child and reads its exit status (0 to 4, equal to the child number). Collecting the status lets the kernel release the PCB of each terminated child, so no zombies remain.

5. Evaluation: Where the Time Goes with 2,000 Active Processes

5.1 Cost breakdown

Each of the 2,000 sessions is a separate process with its own PCB and its own address space. Table 4 lists the costs that grow with the number of processes n. This analysis is reasoned from the operating-system mechanisms above and from the measurements in Section 7; it is not the result of a 2,000-session test on the real server.

Table 4: Cost of running n separate processes

Cost	Why it grows with n	Effect on the examination server
Memory per process	Every process has a private page table, stack, heap and kernel data (PCB and kernel stack). Only read-only code pages can be shared.	Total memory grows as n × (memory per process). If it exceeds RAM, the OS swaps pages out and sessions slow down sharply.
Context-switch cost	Each switch saves and restores registers and changes to another address space, which loses useful cache and TLB contents.	With many runnable processes, each one waits for the others' time slices before it runs again, so response time grows.
Waiting for I/O	All sessions, the auto-save processes and the video proctoring processes share the same disk and network.	Requests queue at the device. A session in the Waiting state can wait a long time and appears to hang.
Process creation burst	At login time 2,000 fork() calls happen together, and each copies the parent's page tables.	The server is busy creating processes just when users need responses.
Kernel bookkeeping	The kernel keeps a PCB, process-table entry and run-queue entry for every process.	More scheduling work and kernel memory per decision as n increases.

Illustration with assumed values (not measured). Suppose each session process uses 50 MB of resident memory. Then 2,000 sessions need 2,000 × 50 MB = 100,000 MB, about 98 GB. A server with less RAM than this would start swapping. Similarly, if a time slice is 1 ms and one context switch costs 5 µs, the direct switching overhead is 5 / (1,000 + 5), about 0.5 percent; the

larger cost of switching between processes comes from the cache and TLB contents that are lost, which this simple calculation does not include.

5.2 Is one process per student a sound design?

Strengths. A separate address space gives strong isolation: a crash or bug in one student's session cannot corrupt another student's answers. The design is also simple to write and to reason about.

Weaknesses. Memory use is n times the memory of one session, context switching between processes is more expensive than between threads, and a login burst creates thousands of processes at once. With I/O queues also growing, these effects together explain how a server can become slow and let some sessions appear to hang.

Judgement. One process per student is sound for a small number of users, but it is not a sound design for 2,000 simultaneous sessions on one server unless the number of processes is limited. The next section chooses a design that keeps the isolation benefit while bounding the cost.

6. Design Decision

Three designs are compared in Table 5: one process per student, one thread per student, and a fixed pool of worker processes.

Table 5: Comparison of the three designs

Criterion	Process per student	Thread per student	Fixed pool of worker processes
Memory	Highest: private address space per session.	Lower: shared address space, one stack per session.	Bounded: fixed number of workers, small state per session.
Context switching	Most expensive: address-space change.	Cheaper between threads of one process.	Number of runnable workers is bounded.
Fault isolation	Best: a crash affects one session.	Weakest: a fault can kill the whole server process.	Good: a crash affects only the sessions of one worker.
Behaviour under a login burst	2,000 forks at once.	2,000 thread creations at once.	No creation burst: requests queue for a free worker.
Programming effort	Simple.	Needs synchronisation of shared data.	Moderate: queue and session-state handling.

Decision. Use a fixed pool of worker processes. Each worker serves many sessions (using threads or an event loop inside the worker), and auto-saving and video proctoring run as their own separate worker processes. Session state and saved answers are kept in the database, not only in worker memory.

Defence.

- Memory is bounded by the number of workers instead of growing with the number of students, which removes the memory-exhaustion and swapping problem.
- The number of runnable processes, and therefore the context-switch overhead, stays bounded however many students log in.
- No fork() is performed per login, so a login burst only lengthens a queue; it does not create thousands of processes.
- A crash is contained within one worker, and because answers are saved in the database, the sessions of that worker can be recovered by another worker.

Trade-offs and limits. Isolation between sessions in the same worker is weaker than with one process per student. The pool size must be tuned by measurement on the real server, and a long-running request can occupy a worker. These limits are accepted because they are far less harmful than exhausting memory with 2,000 processes.

7. Alternative Concept and Complexity Analysis

7.1 How the solution changes

The alternative concept is to run each student session as a thread inside one server process, instead of as a separate process. The server would then create a thread instead of a process at login. All threads share one address space, so the per-session state is only a stack and a small session record.

Pseudocode 1: One thread per student session

```
FUNCTION handle_session(session)
    WHILE session is active
        request <- wait for the next request from the student
        read the question or store the answer
    END WHILE
END FUNCTION

FOR EACH student login
    create a thread that runs handle_session(student)
END FOR
```

7.2 Complexity comparison

Creating n sessions takes O(n) time and O(n) memory in both designs, because each session needs its own task and its own memory. The designs differ in the constants. Let c_p and c_t be the creation time and m_p and m_t the memory of one process-session and one thread-session. Time is $n \times c_p$ versus $n \times c_t$, and memory is $n \times m_p$ versus $n \times m_t$, with $c_p > c_t$ and $m_p > m_t$. A single context switch is O(1) in both designs, and the Linux scheduler selects the next task from a sorted structure in O(log n). A fixed pool of w workers needs only O(w) workers plus O(n) small session records.

Table 6: Process per session compared with thread per session

Criterion	Process per session	Thread per session
Address space	Private to each session.	Shared by all sessions of the server.
Memory per session	Private page tables, heap and stack, plus kernel data (m_p).	A stack plus kernel data; heap and code are shared ($m_t < m_p$).
Creation	fork() copies the page tables of the parent, so cost depends on the memory mapped.	No address space is copied.
Context switch	Changes the address space, so cache and TLB contents are lost.	No address-space change between threads of one process.
Communication	Needs IPC such as pipes or shared memory.	Shared variables, which need locks.
Effect of a crash	Only that session ends.	A fault in one thread ends the whole process and every session.

7.3 Measured creation time

Listing 2 (Appendix A) creates and collects 2,000 short-lived tasks one after another, first with fork() and waitpid(), then with pthread_create() and pthread_join(). It ran three times in the environment of Table 1, which has one processor.

Table 7: Measured creation time of 2,000 tasks (three runs)

Run	Processes: total	Processes: each	Threads: total	Threads: each
Run 1	203.5 ms	101.7 µs	49.4 ms	24.7 µs
Run 2	183.6 ms	91.8 µs	35.7 ms	17.8 µs
Run 3	187.8 ms	93.9 µs	34.9 ms	17.5 µs

In every run, creating and collecting a process took roughly four to five times as long as creating and collecting a thread. This agrees with the expectation that fork() has to set up a separate

address space while a thread shares the existing one. The measurement covers only creation of empty tasks, one at a time. It does not measure memory per session, context switching with 2,000 active sessions, or I/O waiting, so it supports only the creation-time row of Table 6.

7.4 Effect of one session crashing

Listing 3 (Appendix A) makes one session perform an invalid memory access, once as a child process and once as a thread. The results are shown in Output 2.

Output 2: Crash in a child process and in a thread

```
$ ./crash_demo process
session process killed by signal 11; server is still running
$ echo $?
0
$ ./crash_demo thread
Segmentation fault
$ echo $?
139
```

When the session was a separate process, the parent survived, reported that the child was killed by signal 11 (SIGSEGV) and could carry on serving other students. When the session was a thread, the operating system terminated the whole process, shown by the Segmentation fault message and exit code 139 (128 + 11), and the line after pthread_join() was never reached. With one thread per student, a single faulty session would therefore end all 2,000 sessions. This is the main reason the final design of Section 6 keeps worker processes instead of using one server process.

7.5 Conclusion of the comparison

Threads reduce memory, creation time and switching cost compared with processes, but they remove fault isolation. The processes-per-student design is safest but too costly at 2,000 sessions. The fixed pool of worker processes, each running many sessions, gives bounded memory and bounded switching while still limiting a crash to one worker, and is therefore the recommended design.

Appendix A: Source Code Used for the Measurements

Both programs were compiled with gcc -std=c11 -Wall -Wextra -pthread and produced no warnings.

Listing 2: bench.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/wait.h>

#define N 2000

static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

static void *worker(void *arg) { (void)arg; return NULL; }

int main(void)
{
    double t0 = now();
    for (int i = 0; i < N; i++) {
        pid_t pid = fork();
        if (pid == 0) _exit(0);
        waitpid(pid, NULL, 0);
    }
    double tp = now() - t0;

    t0 = now();
    for (int i = 0; i < N; i++) {
        pthread_t t;
        pthread_create(&t, NULL, worker, NULL);
        pthread_join(t, NULL);
    }
    double tt = now() - t0;

    printf("%d x (fork + wait):           %.1f ms total, %.1f us each\n",
           N, tp * 1e3, tp / N * 1e6);
    printf("%d x (pthread_create + join): %.1f ms total, %.1f us each\n",
           N, tt * 1e3, tt / N * 1e6);
    return 0;
}
```

Listing 3: crash_demo.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <pthread.h>
#include <sys/wait.h>

static void *bad_thread(void *arg)
{
    (void)arg;
    volatile int *p = NULL;
    *p = 1;                             /* invalid memory access */
    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: %s process|thread\n", argv[0]);
        return 2;
    }

    if (strcmp(argv[1], "process") == 0) {
        pid_t pid = fork();
        if (pid == 0) {
            volatile int *p = NULL;
            *p = 1;                     /* invalid memory access */
            _exit(0);
        }
        int status;
        waitpid(pid, &status, 0);
        if (WIFSIGNALED(status))
            printf("session process killed by signal %d; "
                   "server is still running\n", WTERMSIG(status));
    } else {
        pthread_t t;
        pthread_create(&t, NULL, bad_thread, NULL);
        pthread_join(t, NULL);
        /* never reached: the whole process dies first */
        printf("session thread finished; server is still running\n");
    }
    return 0;
}
```